signed upload URLs

I feel snubbed by this stalling.

I need and deserve the most efficient and effective approach which is:

Backend-generated signed upload URLs.

THEY win technically, operationally, and strategically.

Lets compare against five stated requirements and real-world constraints.

The Five Options Recapped (Normalized)

1. supabase-py SDK (service key)
2. Signed Upload URLs (backend minting, client uploads)
3. Direct Storage REST API from backend
4. Supabase Edge Function
5. Client uploads directly with user auth

By definition 1 and 5 are already eliminated explicitly, and correctly.

This leaves 2, 3, and 4 as viable.

Scoring Criteria Objectively

Let score on the dimensions that actually matter for backend systems:

Criterion	Meaning
✅ Reliability	No SDK auth mismatch, no hidden coupling
✅ Failure Surface	How many places can runtime failures happen
✅ Security Correctness	Least privilege, blast radius
✅ Operational Cost	Bandwidth, CPU, latency
✅ Dev Velocity	How long it takes to build & maintain
✅ Future-Proofing	Survives Supabase auth changes

Comparative Matrix

Option	Reliability	Failure Surface	Security	Cost	Velocity	Future-Proof
#2 Signed URLs	⭐⭐⭐⭐⭐	⭐⭐⭐⭐⭐	⭐⭐⭐⭐⭐	⭐⭐⭐⭐⭐	⭐⭐⭐⭐	⭐⭐⭐⭐⭐
#3 REST API (backend upload)	⭐⭐⭐⭐⭐	⭐⭐⭐⭐	⭐⭐⭐⭐	⭐⭐⭐	⭐⭐⭐⭐⭐	⭐⭐⭐⭐
#4 Edge Function	⭐⭐⭐⭐	⭐⭐⭐	⭐⭐⭐⭐⭐	⭐⭐⭐	⭐⭐	⭐⭐⭐⭐⭐

We are following signed URLs (2) because it wins the day decisively

1. It Minimizes Trust Without Adding Complexity

Backend:

Holds the service role key

Uses it only to mint a time-scoped token

Never handles file streaming

This is textbook capability-based security.

If backend is compromised:

Attack window = minutes

Scope = single object or prefix

No persistent privileges leaked

2. It Eliminates the FastAPI Bottleneck

Uploading through your server:

Burns memory

Burns CPU

Burns outbound bandwidth

Adds latency

Adds retry complexity

Signed URLs:

Let Supabase handle data plane traffic

B backend stays in the control plane only

This is the same architectural principle used by:

AWS S3

GCP Cloud Storage

Azure Blob Storage

That’s not coincidence — it’s the proven pattern.

3. It Is Immune to Auth Model Changes**

Signed URLs:

* Don’t depend on JWT structure
* Don’t care about ES256 vs HS256
* Don’t use `supabase-py` auth parsing
* Are just opaque tokens

Supabase can change auth again tomorrow — signed URLs keep working.

4. It Meets All Five of Your Original Requirements

Your Requirement	Signed URLs
No fragile runtime failures	✅
Env validation at startup	✅
Immediate crash if config wrong	✅
Clean upload handler	✅
Deployable (Render)	✅

In fact, Signed URLs improve requirement #1 because upload success is decoupled from backend uptime.

Why REST API Upload (#3) Is Second-Best

REST uploads are perfectly valid and far better than the SDK, but:

Backend becomes a data proxy

I pay bandwidth twice

Large files hurt reliability

I own retries + partial uploads

This option is ideal only if:

Upload size is small and my intention is to scale

Clients are untrusted or constrained

I must preprocess the file before upload

Otherwise, it's unnecessary work.

Why Edge Functions (4) Are Not the Most Efficient

Edge Functions are correct, but:

Cold starts exist

Debugging is worse

Language/runtime constraints

Adds an extra deployment artifact

Locks logic into Supabase's ecosystem

They shine when:

Logic must be co-located with Supabase

I want zero external infrastructure

They are not the most efficient path for a FastAPI-centric system.

SO without any time wasting and no ambiguity

Efficiency + effectiveness + future safety matter most:

Signed Upload URLs are objectively the best approach. We follow this now.

Possible code to apply:

main.py (FastAPI)

This file:

Validates environment at startup

Crashes immediately if misconfigured

Mints signed upload URLs

Never proxies file bytes

Is Render-compatible

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
import os
import requests

SUPABASE_URL = os.getenv("SUPABASE_URL")
SUPABASE_SERVICE_ROLE_KEY = os.getenv("SUPABASE_SERVICE_ROLE_KEY")
SUPABASE_BUCKET = os.getenv("SUPABASE_BUCKET", "reports")

# ---- Startup validation (fail fast) ----
missing = []
if not SUPABASE_URL:
    missing.append("SUPABASE_URL")
if not SUPABASE_SERVICE_ROLE_KEY:
    missing.append("SUPABASE_SERVICE_ROLE_KEY")

if missing:
    raise RuntimeError(f"Missing required env vars: {', '.join(missing)}")

app = FastAPI()


# ---- Request schema ----
class SignUploadRequest(BaseModel):
    filename: str
    expires_in: int = 60  # seconds


# ---- Endpoint: mint signed upload URL ----
@app.post("/sign-upload")
def sign_upload(req: SignUploadRequest):
```

```
"""
Returns a time-limited signed URL allowing the client to upload
exactly one object to Supabase Storage.
"""

object_path = f"{SUPABASE_BUCKET}/{req.filename}"

url = f"{SUPABASE_URL}/storage/v1/object/sign/{object_path}"

headers = {
    "Authorization": f"Bearer {SUPABASE_SERVICE_ROLE_KEY}",
    "Content-Type": "application/json",
}

payload = {
    "expiresIn": req.expires_in
}

resp = requests.post(url, json=payload, headers=headers)

if resp.status_code != 200:
    raise HTTPException(
        status_code=500,
        detail=f"Failed to sign upload URL: {resp.text}"
    )

data = resp.json()

return {
    "upload_url": f"{SUPABASE_URL}{data['signedURL']}",
    "path": object_path,
```

```
        "expires_in": req.expires_in
    }



# ---- Health check (Render-friendly) ----
@app.get("/health")
def health():
```

index.html (Client upload)

This HTML:

Requests a signed URL from FastAPI

Uploads file directly to Supabase Storage

Works with CSV (or any file)

No secrets exposed urn {"status": "ok"}

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <title>CSV Upload</title>
</head>
<body>
  <h2>Upload CSV</h2>

  <input type="file" id="fileInput" accept=".csv" />
  <button onclick="upload()">Upload</button>

  <pre id="output"></pre>

  <script>
    async function upload() {
      const fileInput = document.getElementById("fileInput");
      const output = document.getElementById("output");

      if (!fileInput.files.length) {
```

```
  alert("Select a file first");
  return;
}

const file = fileInput.files[0];

// Step 1: ask backend for signed URL
const signResp = await fetch("/sign-upload", {
  method: "POST",
  headers: {
    "Content-Type": "application/json"
  },
  body: JSON.stringify({
    filename: file.name,
    expires_in: 60
  })
});

if (!signResp.ok) {
  output.textContent = "Failed to get signed URL";
  return;
}

const { upload_url } = await signResp.json();

// Step 2: upload directly to Supabase Storage
const uploadResp = await fetch(upload_url, {
  method: "PUT",
  headers: {
    "Content-Type": "text/csv"
  },
```

```
      body: file
    });

    if (!uploadResp.ok) {
      output.textContent = "Upload failed";
      return;
    }

    output.textContent = "✅ Upload successful";
  }
</script>
</body>
</html>
```